Lab 4
Advanced Data Manipulation

CSE 4508

Relational Database Management System Lab

October 13, 2025

Contents

1 Regex, Rollup and Cubes

1.1 Regular Expressions in SQL

Regular expressions allow us to search for complex patterns in string data. Common use cases include extracting or filtering data based on specific patterns.
Example: Extracting a 4-digit year from string data using `REGEXP_SUBSTR`.

```
-- Create and insert data into table
DROP TABLE t1;
CREATE TABLE t1 (data VARCHAR2(50));
INSERT INTO t1 VALUES ('FALL 2014');
INSERT INTO t1 VALUES ('2014 CODE-B');
INSERT INTO t1 VALUES ('CODE-A 2014 CODE-D');
INSERT INTO t1 VALUES ('ADSHLHSALK');
INSERT INTO t1 VALUES ('FALL 2004');
COMMIT;

-- Select rows containing the year >= 2014
SELECT *
FROM t1
WHERE TO_NUMBER(REGEXP_SUBSTR(data, '\d{4}')) >= 2014;
```

Input:

```
FALL 2014
2014 CODE-B
CODE-A 2014 CODE-D
ADSHLHSALK
FALL 2004
```

Output:

```
FALL 2014
2014 CODE-B
CODE-A 2014 CODE-D
```

1.2 ROLLUP for Subtotals

The `ROLLUP` operator generates subtotals for specified columns in a `GROUP BY` query.
Example: Calculating subtotals for sales by department.

```
-- Generate subtotals using ROLLUP
SELECT department, SUM(sales) AS total_sales
FROM order_details
GROUP BY ROLLUP(department)
ORDER BY department;
```

Output:

```
Department      Total Sales
Clothing        1100
Electronics     2700
Furniture       2900
                6700  -- Grand Total
```

1.3 CUBE for Cross-Tabulation

The CUBE operator calculates subtotals for all possible combinations of grouping columns.

Example: Counting employees by department and designation using CUBE.

```
-- Drop table if it exists
DROP TABLE employees;

-- Create table employees with department names
CREATE TABLE employees (
    employee_id NUMBER,
    department VARCHAR2(50),
    designation VARCHAR2(50),
    hire_date DATE
);

-- Insert data into employees
INSERT INTO employees (employee_id, department, designation,
   hire_date)
VALUES (1, 'Sales', 'Manager', TO_DATE('15-02-2005', 'DD-MM-YYYY'))
   ;

INSERT INTO employees (employee_id, department, designation,
   hire_date)
VALUES (2, 'Sales', 'Manager', TO_DATE('21-01-2007', 'DD-MM-YYYY'))
   ;

INSERT INTO employees (employee_id, department, designation,
   hire_date)
VALUES (3, 'Human Resources', 'Assistant', TO_DATE('10-05-2010', '
   DD-MM-YYYY'));

-- Commit the transaction
COMMIT;
```

```
-- Count employees with subtotals for all combinations
SELECT department, designation, COUNT(*) AS total
FROM employees
GROUP BY CUBE(department, designation)
ORDER BY department, designation;
```

Output:

```
Department          Designation     Total
Human Resources     Assistant        1
Human Resources                      1
Sales               Manager          2
Sales                                2
                                     3  -- Grand Total
```

2 OLAP, Data Cube, and Hierarchical Aggregation

2.1 OLAP (Online Analytical Processing)

OLAP is a powerful approach for analyzing multidimensional data interactively. It allows users to view data from different perspectives and perform complex queries with ease, often through operations such as slicing, dicing, rollups, and drill-downs.

Example: Consider a sales dataset with dimensions like `Product`, `Region`, and `Time`. Using OLAP, we can generate reports that break down sales by different regions and product categories, or analyze trends over time.

```
-- OLAP Query Example: Sales by Product, Region, and Year
SELECT product, region, EXTRACT(YEAR FROM sales_date) AS year, SUM(
   sales)
FROM sales_data
GROUP BY CUBE(product, region, EXTRACT(YEAR FROM sales_date))
ORDER BY product, region, year;
```

2.2 Data Cube

A Data Cube extends the concept of traditional aggregation in SQL by enabling cross-tabulation for multiple dimensions. The cube operator generates subtotals for all combinations of the specified columns.
Example: Using the `CUBE` operator to compute subtotals for sales across the dimensions of `Product`, `Region`, and `Year`.

```
-- CUBE Example: Sales by Product, Region, and Year
SELECT product, region, EXTRACT(YEAR FROM sales_date) AS year, SUM(
   sales)
FROM sales_data
GROUP BY CUBE(product, region, EXTRACT(YEAR FROM sales_date));
```

Output:

```
Product     Region       Year    Sales
Laptop      North        2020    15000
Laptop      North                15000
Laptop                   2020    15000
                        2020    50000
                                 80000  -- Grand Total
```

2.3 Extended Aggregation and Calculated Measures

Extended aggregation refers to advanced aggregation techniques that go beyond simple `SUM` or `COUNT`. These aggregations may include statistical functions, moving averages, or user-defined metrics.
Example: Calculating the average sales and variance for each product.

```
-- Extended Aggregation Example: Average and Variance of Sales by
   Product
SELECT product, AVG(sales) AS avg_sales, VARIANCE(sales) AS
   sales_variance
FROM sales_data
GROUP BY product;
```

Output:

```
Product     Avg Sales   Sales Variance
Laptop      15000       250000
Phone       12000       144000
Tablet      9000        81000
```

2.4 Hierarchy of Dimensions

In OLAP and Data Cube operations, dimensions often have a hierarchical structure that allows users to drill down or roll up through different levels of aggregation. For instance, a `Time` dimension may have a hierarchy of `Year` → `Quarter` → `Month`.
Example: Drilling down from the year to the quarter level in a time hierarchy.

```
-- Hierarchical Drill-Down: Sales by Year, Quarter
SELECT EXTRACT(YEAR FROM sales_date) AS year,
       EXTRACT(QUARTER FROM sales_date) AS quarter,
       SUM(sales) AS total_sales
FROM sales_data
GROUP BY ROLLUP(EXTRACT(YEAR FROM sales_date), EXTRACT(QUARTER FROM
    sales_date));
```

Output:

```
Year    Quarter    Total Sales
2020    1          25000
2020    2          27000
2020               52000
2021    1          30000
2021               30000
                   82000  -- Grand Total
```

3 Date Operations in Oracle SQL

Dates in Oracle can be formatted and manipulated using various built-in functions. Below, we explore some common operations involving date formatting and date arithmetic.

3.1 Formatting Dates with `TO_CHAR`

The `TO_CHAR` function is used to convert a date to a string in a specified format. In this case, we format the hire dates from the `employees` table.

```
-- Format hire dates
SELECT employee_id, TO_CHAR(hire_date, 'DD-MM-YYYY') AS
   formatted_date
FROM employees;
```

Output:

Employee ID	Formatted Date
1	15-02-2005
2	21-01-2007
3	10-05-2010

Table 1: Formatted Hire Dates

3.2 Date Arithmetic with `ADD_MONTHS`

Oracle provides the `ADD_MONTHS` function, which allows adding or subtracting a number of months from a given date. Below is an example where we calculate the date 6 months after each employee's hire date.

```
-- Add 6 months to hire dates
SELECT employee_id, hire_date, ADD_MONTHS(hire_date, 6) AS new_date
FROM employees;
```

Output:

Employee ID	Hire Date	New Date (6 months later)
1	15-02-2005	15-08-2005
2	21-01-2007	21-07-2007
3	10-05-2010	10-11-2010

Table 2: Hire Dates and Corresponding Dates 6 Months Later

3.3 Calculating the Difference Between Dates

To calculate the difference between two dates in Oracle, the subtraction operator can be used. The difference is returned as the number of days.

```
-- Calculate the number of days since each employee was hired
SELECT employee_id, hire_date, SYSDATE - hire_date AS
   days_since_hired
FROM employees;
```

Output:

Employee ID	Hire Date	Days Since Hired
1	15-02-2005	7143
2	21-01-2007	6241
3	10-05-2010	5268

Table 3: Days Since Employees Were Hired

4 Tasks

1. Write a query to select rows from `t1` where the year extracted with a regular expression is greater than or equal to 2010.
2. Create a PL/SQL block that retrieves all rows from `t1`, extracts the year using a regular expression, and prints it using `DBMS_OUTPUT`.
3. Write a query to group sales data by department and show only those departments where the total sales exceed 1200.
4. Create a PL/SQL block to group sales data by department and print departments where the total sales are over 1000.
5. Use the `ROLLUP` function to calculate subtotals for sales data in the `order_details` table.
6. Write a PL/SQL block that uses the `ROLLUP` function and prints the subtotals for each department.
7. Write a query to generate cross-tabular data using `CUBE` for the `employees` table based on department and designation.
8. Write a query to format the hire dates of all employees in the `employees` table to 'DD-MM-YYYY'.
9. Create a PL/SQL block that prints the difference in years between the earliest and latest hire dates from the `employees` table.

Submission Guidelines

You are required to submit a report that includes your code, a detailed explanation, and the corresponding output. Rename your report as `<StudentID_Lab_1.pdf>`.

- Your lab report must be generated in LaTeX. Recommended tool: Overleaf. You can use some of the available templates in Overleaf.
- Include all code/pseudo code relevant to the lab tasks in the lab report.
- Please provide citations for any claims that are not self-explanatory or commonly understood.
- It is recommended to use vector graphics (e.g., `.pdf`, `.svg`) for all visualizations.
- In cases of high similarities between two reports, both reports will be discarded.